

Reproducing a Tuning-Free Plug-and-Play algorithm for Sparse-View CT

Research project report presented by

Emilia Zabrzanska

Department

Department of Physics (Cavendish Laboratory)

Degree

Master of Philosophy in Data Intensive Science



Magdalene College
University of Cambridge
July 2026

Submitted in partial fulfillment of the requirements for the
Master of Philosophy in Data Intensive Science

Declaration

I, Emilia Zabrzanska of Magdalene College, being a candidate for the Master of Philosophy in Data Intensive Science, hereby declare that this report and the work described in it are my own work, except where specified below and in the AI/LLM usage declaration (Section A.4).

This report has a word count of approximately **7,000** words, counted in line with the course's word-count policy, including the abstract, tables, footnotes, and appendices, and excluding the table of contents, figure and table captions, lists of figures and tables, equations, the bibliography, the acknowledgements, and the auto-generation usage appendix (Section A.4).

The reproduction of the Tuning-Free Plug-and-Play algorithm [?] was carried out within the LION toolbox, developed by the Cambridge Image Analysis group. A pull request contributing improvements to LION's FBPCovNet implementation was submitted as part of this work (PR #184 on the CambridgeCIA/LION repository).

Signed: Emilia Zabrzanska

Date: July 2026

Acknowledgements

Abstract

Sparse-view computed tomography (CT) reduces patient radiation dose by acquiring fewer projection angles, at the cost of creating a more ill-posed reconstruction problem. The Tuning-Free Plug-and-Play (TFPnP) method of ? addresses this by using reinforcement learning to select ADMM hyperparameters automatically. This project reproduces TFPnP within the LION toolbox, targeting 30-view parallel-beam CT on the LIDC-IDRI dataset with the natural-image DRUNet as a prior. The reproduction achieves 27.27 dB PSNR at 5% sinogram noise, an improvement of +2.96 dB over hand-calibrated Fixed PnP-ADMM, confirming the paper’s central claim about the advantage of learned hyperparameter policies.

The evaluation also reveals a partial reversal of the paper’s TFPnP/FBPCNN baseline comparison, as with FBPCNN trained end-to-end on the target CT domain, it outperforms TFPnP by +1.31 dB at 5% noise, but TFPnP overtakes on all three metrics at 10% noise, revealing the noise-robustness advantage of the iterative plug-and-play architecture. An extension replacing the natural-image DRUNet with a CT-specific variant produces a +0.71 to +1.98 dB improvement in Fixed PnP-ADMM, growing with noise, but a full TFPnP policy retrain against the CT-DRUNet was destabilised by RL training difficulties that future work is needed to resolve. All divergences from the paper, including choices left underspecified, are consolidated in a single table with appropriate justifications. The reproduction is released as a working Python package integrated with LION, together with an upstream contribution fixing a bug in the toolbox’s FBPCNN baseline.

Contents

1	Introduction	1
1.1	Contributions	2
1.2	Report structure	2
2	Background and Related Work	3
2.1	Sparse-view CT	3
2.2	Classical reconstruction	4
2.3	Two-step reconstruction methods	4
2.4	Plug-and-play priors and ADMM	5
2.5	DRUNet	5
2.6	Tuning-free hyperparameter control with reinforcement learning	6
3	Design and Implementation	7
3.1	Overview	7
3.2	Geometry, data, and noise conventions	7
3.3	TFPnP policy and RL training	9
3.4	ADMM step and z-step gradient normalisation	10
3.5	Baselines	10
3.6	CT-specific DRUNet training	11
3.7	Compute infrastructure	12
3.8	Evaluation metrics	14
3.9	Reproducibility difficulties	14
4	Evaluation	15
4.1	Headline reproduction result	15
4.2	Cross-noise behaviour	16
4.3	Per-image variance	17
4.4	Hyperparameter calibration sweeps	17
4.5	In-domain denoiser experiment	19
4.6	TFPnP policy retrain with CT-DRUNet	20
4.7	Reward shaping ablations	21
5	Discussion	23

5.1	Contextualising the TFPnP–FBPConvNet comparison	23
5.2	Does an in-domain denoiser close the gap?	23
5.3	On the choice of evaluation metrics	24
5.4	On direct comparison with ?	24
5.5	Broader implications	25
6	Conclusion	27
A	Supplementary Material	28
A.1	Compute infrastructure	28
A.2	Run matrix	28
A.3	Repository and artefacts	29
A.4	AI/LLM usage	30

List of Figures

2.1	The sparse-view CT reconstruction problem, shown on noisy and clean images. .	4
2.2	DRUNet architecture. The noisy image and constant-valued noise map σ are concatenated along the channel dimension and passed through a head convolution, three encoder stages of two residual blocks each (with strided convolutions between stages for downsampling), a four-block bottleneck, three decoder stages of two residual blocks each (with transposed convolutions between stages for upsampling), and a tail convolution. Skip connections carry encoder features directly to the corresponding decoder stage. Adapted from ?.	6
3.1	A lung slice from the LIDC-IDRI training split, in native μ units (approximately $[0, 2.3] \text{ cm}^{-1}$).	8
3.2	CT-DRUNet training curves. Left, MSE loss on the LIDC training split over 50 epochs. Right, validation PSNR at $\sigma \in \{5, 15, 25\}$. Best checkpoint selected by peak validation PSNR at $\sigma = 15$	12
4.1	Reconstruction gallery for run_04 at 5% noise. Rows: best, median, and worst test images by TFPnP PSNR. Per-cell labels give PSNR, SSIM, and HaarPSI. . .	16
4.2	Mean PSNR, SSIM, and HaarPSI vs sinogram noise level. TFPnP overtakes FBPConvNet on all three metrics at 10% noise.	17
4.3	Per-image PSNR at 5% noise. TFPnP's three largest wins (left) and losses (right) versus the next-best non-TFPnP method. Homogeneous slices favour TFPnP, complex ones favour FBPConvNet.	17
4.4	σ sensitivity for natural-image DRUNet within Fixed PnP-ADMM at 5% noise. Peak at $\sigma = 1.5$	18
4.5	σ sensitivity for CT-trained DRUNet within Fixed PnP-ADMM at 5% noise. Peak at $\sigma = 8.0$	18
4.6	μ sensitivity for CT-trained DRUNet at $\sigma = 8$. Peak at $\mu = 10$	19
4.7	Fixed PnP-ADMM with denoisers at their calibrated optimum, at 5%, 7.5%, and 10% noise.	20
4.8	Training curves for run_11	21

List of Tables

3.1	Implementation choices differing from ?	13
4.1	Reconstruction quality on the LIDC-IDRI test set (389 images). Best in bold , second-best <u>underlined</u>	16
4.2	Fixed PnP-ADMM at each denoiser’s calibrated optimum.	19
4.3	Reward-shaping ablation at 5% noise. Best values in bold	22
A.1	Summary of TFPnP training runs. Runs 04–12 use 250 LIDC patients, 80 epochs, and the paper’s default hyperparameters unless noted.	29

Chapter 1

Introduction

Computed tomography (CT) is one of the most widely used diagnostic imaging modalities in modern medicine, providing detailed three-dimensional views of internal anatomy. It is not, however, without cost, as CT scans use ionising X-ray radiation that can be harmful with repeated exposure. Reducing radiation dose while preserving diagnostic image quality is therefore a central concern in clinical practice. The most common strategies are low-dose CT and, the focus of this project, sparse-view CT, where fewer projection angles are acquired per scan to reduce total exposure. The resulting inverse problem is more ill-posed, since with fewer measurements, the reconstruction is under-determined, and standard analytical inversion produces prominent streak artefacts.

Classical sparse-view reconstruction relies on filtered backprojection (FBP) or iterative methods with hand-crafted regularisers such as total variation (TV). FBP is fast but produces the streak artefacts already mentioned, and TV mitigates these but imposes a piecewise-constant prior, penalising differences between neighbouring pixels. Subtle intensity variations are therefore treated the same as noise and get smoothed into uniform patches, losing fine anatomical detail. Learned methods now dominate reconstruction benchmarks, and fall into two main families. Two-step reconstruction methods such as FBPCNN [??] take the FBP reconstruction as input and train a network to refine it into a clean image. Plug-and-play (PnP) methods combine iterative optimisation frameworks such as ADMM with learned image denoisers, allowing a single denoiser to serve as a prior for any inverse problem. The trade-off is that the denoising strength σ and the penalty parameter μ must be tuned carefully per iteration and per image, or performance degrades quickly.

The Tuning-Free Plug-and-Play method (TFPnP) of ? addresses this burden by formulating the choice of σ and μ as a reinforcement learning problem, where a policy network is trained to select these hyperparameters automatically at each ADMM iteration, based on the current state of the reconstruction. Once trained, the policy is able to operate without any per-problem tuning at inference time. ? report state-of-the-art results on several inverse imaging problems including sparse-view CT, where TFPnP outperforms both classical baselines and the end-to-end FBPCNN. Reproducing these results in an independent implementation, and testing their

sensitivity to methodological choices left underspecified in the paper, form the primary focus of this project.

This project reproduces TFPnP from scratch within the LION toolbox [?], a Python framework for AI-driven inverse imaging developed at the University of Cambridge. The reproduction is evaluated on the LIDC-IDRI clinical lung CT dataset [?], using the DRUNet denoiser [?]. Throughout the project, several implementation choices affecting the final result were identified and documented. Two extensions were also carried out, including a domain-adapted CT-DRUNet and a set of reward-shaping ablations to test alternative RL reward signals. The reproduction reveals both the practical strengths of TFPnP’s approach, particularly its robustness across noise levels, as well as its sensitivities to hyperparameter and training-data choices that the original paper does not discuss, including how the choice of RL action ranges interacts with training stability, and how an in-domain denoiser affects reconstruction quality within a plug-and-play pipeline.

1.1 Contributions

The contributions of this project are as follows.

- (i) A faithful reproduction of the TFPnP method within the LION toolbox, targeting 30-view parallel-beam sparse-view CT with the natural-image DRUNet as a prior.
- (ii) A finding that domain-specific FBPCnnNet training substantially changes the ranking between two-step reconstruction and plug-and-play methods relative to the paper’s out-of-domain comparison, with FBPCnnNet leading at 5% noise and TFPnP overtaking on all three metrics at 10% noise.
- (iii) A CT-specific DRUNet variant that improves reconstruction quality by 0.71–1.98 dB PSNR over the natural-image DRUNet in Fixed PnP-ADMM, with the advantage growing as noise increases.
- (iv) A reward-shaping ablation testing four alternative RL reward signals (SSIM, HaarPSI, and mixed rewards), showing that the paper’s default PSNR reward remains a defensible choice.
- (v) An upstream contribution to the LION toolbox in the form of pull request #184, fixing a bug in the FBPCnnNet integration.

1.2 Report structure

Chapter 2 introduces the relevant technical background and positions TFPnP within the wider landscape of sparse-view CT reconstruction methods. Chapter 3 describes the reproduction’s design and implementation in detail, including all deviations from the paper. Chapter 4 presents the empirical evaluation, and Chapter 5 interprets the findings. Chapter 6 summarises the project and outlines directions for future work, and Chapter A provides supplementary material covering compute infrastructure, the full run matrix, and repository pointers.

Chapter 2

Background and Related Work

This chapter provides the technical background needed to understand the reproduction described in Chapters 3–5, and positions TFPnP within the wider landscape of sparse-view CT reconstruction methods. It covers the reconstruction problem itself, classical and learned approaches, the ADMM algorithm and its plug-and-play extension, the DRUNet denoiser, and the reinforcement learning framework used by TFPnP to select ADMM hyperparameters automatically.

2.1 Sparse-view CT

CT reconstruction aims to recover an internal image x of a patient from a set of X-ray measurements y taken at different angles around the body, modelled by the forward equation

$$y = Ax + \epsilon \tag{2.1}$$

where A is the forward operator modelling the X-ray acquisition and ϵ is measurement noise, and the exact form of A depends on the scan geometry. Under the parallel-beam geometry used in this project, A is the discrete Radon transform, mapping the image to its line integrals along parallel rays at each projection angle.

In a full-view acquisition, x can be recovered accurately by inverting A analytically using filtered backprojection (FBP), but in sparse-view conditions, the number of projection angles is reduced, under-sampling the sinogram along ray directions and producing streak artefacts in the FBP reconstruction, which are visible as diagonal or radial patterns aligned with the missing projection angles. Figure 2.1 illustrates the effect.

Because the inverse problem is under-determined under sparse-view conditions, additional constraints, or prior information about what images should look like, are required to recover a high-quality reconstruction. The choice of prior is what distinguishes the reconstruction methods reviewed in the rest of this chapter.

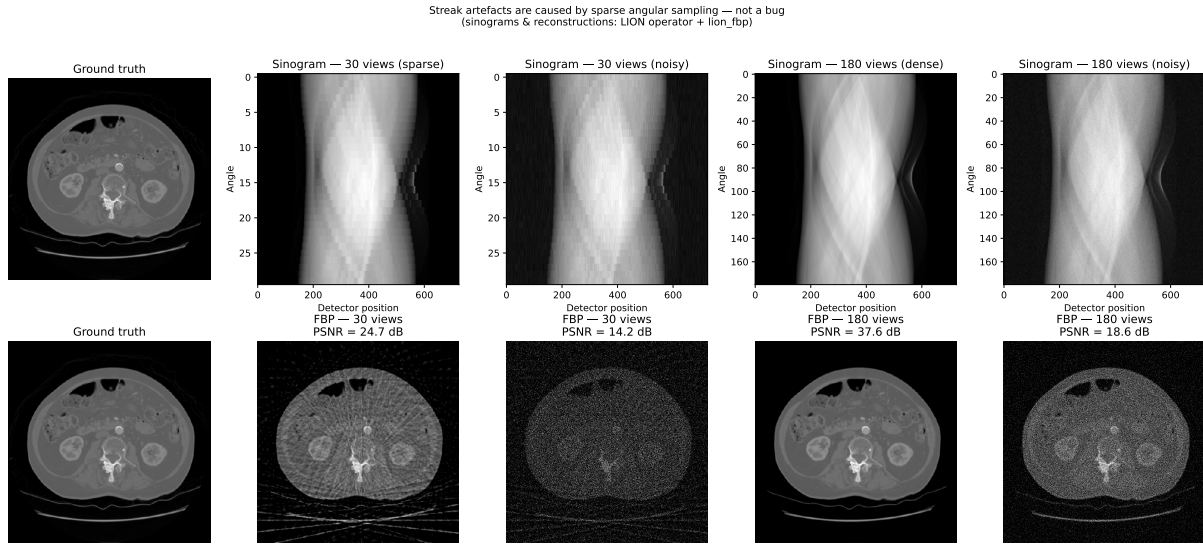


Figure 2.1: The sparse-view CT reconstruction problem, shown on noisy and clean images.

2.2 Classical reconstruction

Filtered backprojection (FBP) is the standard analytical method for CT reconstruction, applying a ramp filter in the frequency domain to compensate for the smoothing effect of backprojection. As introduced above, FBP produces severe streak artefacts under sparse-view conditions because the inversion problem becomes ill-posed.

Total variation (TV) regularisation is the standard iterative baseline for sparse-view reconstruction, formulating the inverse problem as the minimisation of a data-fidelity term together with a TV penalty that encourages piecewise-constant images. TV performs well on images with sharp boundaries and smooth regions, but its prior tends to produce cartoon-like reconstructions in which fine anatomical detail is lost.

2.3 Two-step reconstruction methods

Two-step reconstruction methods [?] take an analytical reconstruction such as FBP as input and train a neural network to refine it into a clean image. Their main advantage is inference speed, as a single forward pass produces a reconstruction almost instantly once the network is trained. The main disadvantage is that the trained network is specialised to the conditions seen during training, and may fail to generalise outside them.

FBPConvNet [?] is the canonical example in this family. It applies a U-Net to a sparse-view FBP reconstruction, mapping it to a clean full-view reconstruction. Taking the FBP reconstruction as input, rather than the raw sinogram, allows the network to operate in the image domain and avoids the need to learn the physics of the inverse problem.

2.4 Plug-and-play priors and ADMM

Plug-and-play (PnP) methods combine iterative optimisation with learned denoisers. Regularised reconstruction combines a data fidelity term measuring how well the reconstruction matches the measurements, and a prior term $R(x)$ encoding prior knowledge about what images should look like. This is known as the variational regularisation (VR) framework, or equivalently the maximum a posteriori (MAP) estimator under a Bayesian formulation, where the data-fidelity term corresponds to the negative log-likelihood and $R(x)$ to the negative log-prior.

$$x^* = \arg \min_x \frac{1}{2} \|Ax - y\|^2 + \lambda R(x) \quad (2.2)$$

The parameter λ balances the two objectives, and common choices for $R(x)$ include total variation, wavelet sparsity, and learned image priors.

The Alternating Direction Method of Multipliers (ADMM) is a standard algorithm for solving Equation (2.2), splitting the problem into three simpler sub-problems that are solved in turn, with an x -step applying the prior (a denoising operation), a z -step enforcing data fidelity, and a u -step tracking the disagreement between x and z across iterations. A penalty parameter μ controls how strongly x and z are coupled.

The plug-and-play insight is that the x -step reduces to a denoising problem, so rather than solving it exactly, it can be replaced by any denoiser D_σ , where σ controls the denoising strength. The denoiser then acts as an implicit prior, and no explicit form for $R(x)$ is required. The same denoiser can be reused across different inverse problems by pairing it with the appropriate data-fidelity term. The remaining difficulty is that both σ and μ must be chosen well, and typically require tuning per iteration and per problem instance, which is the problem TFPnP addresses.

2.5 DRUNet

DRUNet [?] is a residual U-Net designed for plug-and-play priors, and the denoiser used in this project’s TFPnP reproduction. It has 32.6 million parameters organised as a four-stage encoder-decoder with residual blocks and skip connections. Its input is the noisy image stacked with an additional channel holding a constant noise-level value σ , so a single trained network can denoise at any σ in its trained range by setting that channel accordingly. This makes DRUNet well suited to plug-and-play use, where σ typically changes across ADMM iterations. Figure 2.2 shows the architecture.

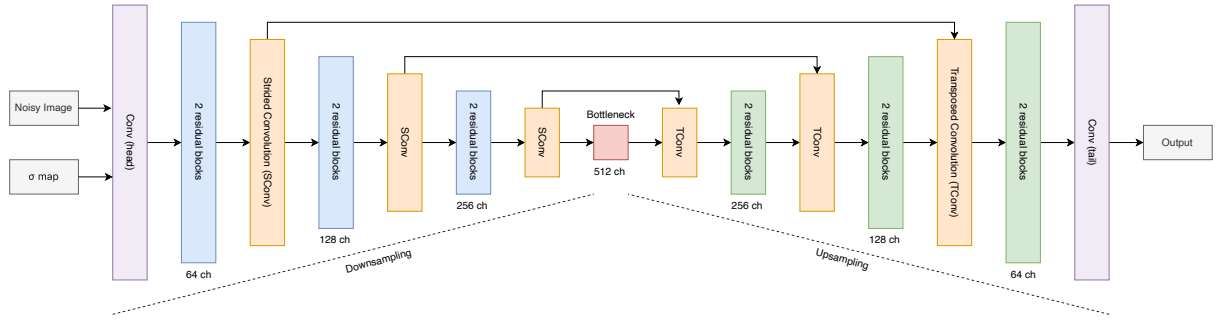


Figure 2.2: DRUNet architecture. The noisy image and constant-valued noise map σ are concatenated along the channel dimension and passed through a head convolution, three encoder stages of two residual blocks each (with strided convolutions between stages for downsampling), a four-block bottleneck, three decoder stages of two residual blocks each (with transposed convolutions between stages for upsampling), and a tail convolution. Skip connections carry encoder features directly to the corresponding decoder stage. Adapted from ?.

DRUNet is trained on 400 natural images from BSD400, with additive Gaussian noise sampled uniformly from $\sigma \in [1, 50]$. The standard grayscale checkpoint (`drunet_gray.pth`) is the reference denoiser used by TFPnP for CT reconstruction and by many other plug-and-play works. Its use in this project is discussed in Section 3.3.

2.6 Tuning-free hyperparameter control with reinforcement learning

TFPNP [?] treats the selection of σ and μ across ADMM iterations as a reinforcement learning problem. At each iteration, a policy network observes the current ADMM state (the variables x , z , u , the noise level, and iteration progress) and outputs both a (σ, μ) prediction for the next m ADMM steps and a decision on whether to terminate. The policy is rewarded at each step by the PSNR gain minus a small constant penalty ($\eta = 0.05$), discourages the policy from taking too many steps. The policy is trained once on a large image dataset, after which no further tuning is required at inference time.

Two sub-policies are trained. The discrete termination policy π_1 uses REINFORCE, a policy-gradient method that reinforces actions leading to higher rewards. The continuous parameter policy π_2 uses deep deterministic policy gradient (DDPG), an actor-critic method backpropagated through the differentiable ADMM environment. Once trained, the policy produces σ and μ automatically at inference.

A notable limitation of ?’s evaluation is that the policy and denoiser are both trained on natural images (PASCAL VOC and BSD400 respectively) and evaluated on medical images (CT slices from the COVID-19 CT dataset), making the entire pipeline out-of-domain relative to the target application. This project evaluates TFPnP against an in-domain FBPCNet on the LIDC-IDRI dataset, and extends TFPnP with a CT-specific DRUNet to assess whether an in-domain denoiser improves reconstruction quality.

Chapter 3

Design and Implementation

3.1 Overview

The reproduction is implemented as a Python package, `ct_tfpnp`, that integrates with the LION toolbox [?] for the CT geometry, forward operator, and dataset handling. The package contains all the components of the TFPnP algorithm, including the ResNet-18 policy network, the residual value network, the ADMM environment, the DRUNet denoiser wrapper, and the RL training loop, as well as the baseline reconstruction methods (FBP, TV, DRUNet alone, FBPCnvNet, and Fixed PnP-ADMM) used for comparison.

The full experimental pipeline covers eleven trained TFPnP runs and two baseline model checkpoints (FBPCnvNet and a CT-trained DRUNet), with each run producing a set of diagnostic figures via `scripts/evaluate_run.py`, including training curves, per-image metrics, policy behaviour, and a reconstruction gallery. All experiments are launched on the Cambridge Service for Data Driven Discovery (CSD3) HPC cluster through SLURM job scripts (see Chapter A), with outputs written to a results directory for analysis.

3.2 Geometry, data, and noise conventions

All experiments use a sparse 30-view parallel-beam geometry, matching ?. As LION had no matching preset, a custom `Geometry` object was constructed with 30 angles uniformly spaced over $[0, \pi)$, a 512×512 image, and 724 detectors ($[512\sqrt{2}]$, chosen to cover the image diagonal). The forward operator is LION’s `CTProjectionOp`, a thin wrapper around `tomosipo` [?], whereas ? use `TorchRadon` [?]. The two are mathematically equivalent but differ in scaling conventions (Section 3.4).

Training and evaluation data come from LIDC-IDRI [?], split at the patient level into 80/10/10 partitions using LION’s default loader. This yields approximately 3300, 401, and 389 slices in the training, validation, and test splits respectively, with up to 5 slices per patient and a 50/50 nodule-to-non-nodule ratio. ? evaluate on 50 randomly selected lung slices from the COVID-19 CT lung and infection segmentation dataset [?], making our test set substantially larger. Figure 3.1 shows a representative training slice.

Image intensities are preserved in linear attenuation coefficient (μ) units throughout the pipeline, with values approximately in $[0, 2.5] \text{ cm}^{-1}$. This convention is set by LION’s LIDC loader via its `from_HU_to_mu` transform, and is preserved rather than rescaled to $[0, 1]$ as in many other reproductions, to retain the physical interpretation of reconstructed values.

Sinogram noise is added following ?’s percentage-of-signal-standard-deviation convention. For each ground truth image, the clean sinogram $y_{\text{clean}} = Ax_{\text{gt}}$ is normalised by a per-image factor $\text{SCALE} = y_{\text{clean}}.\text{max}()/x_{\text{gt}}.\text{max}()$ so that the percentage noise fraction applies consistently across images. Additive Gaussian noise is then applied at 5%, 7.5%, or 10% of the normalised sinogram’s standard deviation, and the result is rescaled back to the original operator range. Noise seeds are deterministic per image (`seed = img_idx * 100`), so paired comparisons between methods use identical realisations.

LION provides a more physically realistic `sinogram_add_noise` routine based on a Poisson-Gaussian model, which would better represent the photon-counting statistics of real CT acquisitions, but it takes physical parameters (photon count, detector gain) rather than a percentage noise fraction, so its noise levels do not map directly onto the paper’s 5%, 7.5%, 10% conditions. The paper’s percentage convention is used here to keep the implementation as close to ?’s as possible and preserve the direct comparability of results.

One consequence of retaining native μ units is that PSNR values in this report are approximately 5 dB higher than those reported in ?, affecting cross-study comparisons but not within-experiment rankings (Section 3.8).

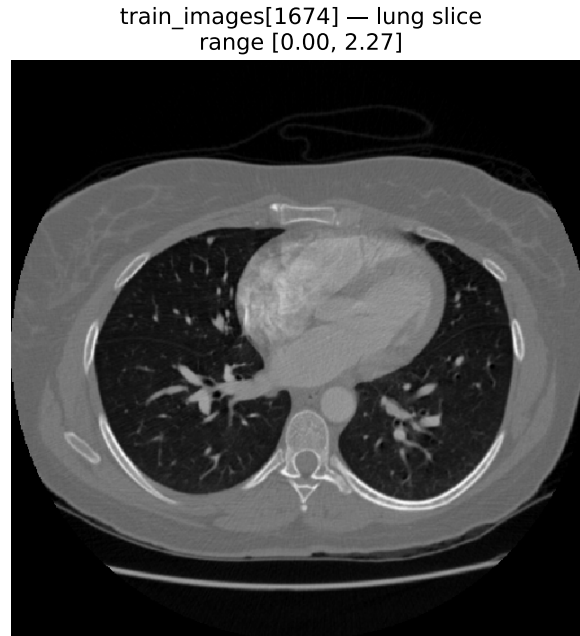


Figure 3.1: A lung slice from the LIDC-IDRI training split, in native μ units (approximately $[0, 2.3] \text{ cm}^{-1}$).

3.3 TFPnP policy and RL training

The TFPnP policy network follows the architecture described by ?. A ResNet-18 backbone extracts features from a five-channel input consisting of the current ADMM iterates (x, z, u) , a noise-level map, and an iteration-progress map. Two output heads share the resulting 512-dimensional feature vector, a two-way termination head predicting whether to stop, and a parameter head predicting the next $m = 5$ pairs of (σ, μ) via a sigmoid activation rescaled to the configured action ranges.

The backbone follows torchvision’s ResNet-18, with only the first convolutional layer replaced to accept five input channels rather than three. Table 1 of ? specifies a 3×3 stem, whereas torchvision’s default is 7×7 , which is retained for this implementation. Our slices are 512×512 , so layer4 emits 16×16 rather than 4×4 , but an adaptive average pool absorbs the difference. Both deviations are recorded in Table 3.1.

Table 1 gives conv1 an output of 64×64 and layer1 an output of 32×32 , but does not state how this is obtained, since a standard ResNet-18’s layer1 does not downsample. We retain torchvision’s max-pooling layer, which reproduces the stated output sizes.

The value network departs from the paper. The caption of Table 1 specifies the same feature extractor as the policy, with batch normalisation replaced by weight normalisation and ReLU replaced by TReLU. Our value network is instead a residual network of eight blocks at constant width 64, with no normalisation, standard ReLU activations, no spatial downsampling, and a final linear layer mapping a 64-dimensional feature vector to the scalar value estimate. It preserves the paper’s removal of batch normalisation but not the replacements specified alongside it. The consequences of this divergence are analysed in Section 5.2.

Policy action ranges are $\sigma \in [1, 5]$ and $\mu \in [10, 100]$. The σ range is calibrated to the natural-image DRUNet’s useful operating region for CT, identified from a preliminary sensitivity sweep (Section 4.4) in which $\sigma > 5$ over-smoothed reconstructions. The μ range balances against the normalised data-fidelity gradient (Section 3.4), with ?’s corresponding effective range set at $[0.01, 1]$ because their unnormalised gradient is $\sim 100\times$ smaller.

The RL formulation matches ?’s equations 15–17. The value network is trained on a temporal-difference (TD) loss, which regresses each state’s value estimate towards the reward received plus the discounted estimate of the next state, with discount factor $\gamma = 0.99$. Stability is maintained by comparing against a target network, a slowly-updated copy of the value network, refreshed by exponential moving average at rate 10^{-3} . The termination policy π_1 uses a value baseline, subtracting the value network’s prediction from the reward before applying the REINFORCE update, which reduces gradient variance. The reward is $r = \Delta\text{PSNR} - \eta$ with $\eta = 0.05$.

Learning rates are 10^{-4} for the critic, 3×10^{-5} for the shared policy features and termination head, and 10^{-6} for the parameter head, the last kept low to compensate for the sensitivity of π_2 ’s gradient through the ADMM rollout. The paper’s hyperparameters $m = 5$, $N = 6$, and $\eta = 0.05$ are preserved exactly.

3.4 ADMM step and z-step gradient normalisation

The ADMM environment implements ?’s equations 7–9, with each outer iteration performing three sub-steps. The x -step applies the DRUNet denoiser as an implicit image prior, the z -step enforces data fidelity against the noisy sinogram, and the u -step updates the dual variable to accumulate constraint violation.

The z -step is solved by six iterations of gradient descent, so that the update is differentiable end-to-end. The model-based DDPG update for π_2 requires backpropagation through the entire z -step, which gradient descent supports through automatic differentiation.

LION’s adjoint operator $A^\top$ implements the raw backprojection of tomosipo’s discrete Radon transform, summing sinogram values along ray paths without normalisation, so $A^\top(Az)$ produces per-voxel values much larger than the input image. Without adjustment, the data-fidelity gradient $A^\top(Az - y)$ would dominate the proximal term $\mu(z - \text{target})$ for any reasonable μ , rendering the policy’s μ predictions meaningless. To resolve this, the backprojected residual is normalised to unit maximum before being combined with the proximal term,

$$\text{grad} = \frac{A^\top r}{\|A^\top r\|_\infty + \epsilon} + \mu(z - \text{target}), \quad (3.1)$$

where $r = Az - y$ is the sinogram residual and $\epsilon = 10^{-8}$ prevents division by zero. The proximal term pulls z towards the current ADMM estimate $\text{target} = x + u$, with μ controlling the strength. Normalising the first term places its magnitude on the same scale as the proximal term, so μ can meaningfully balance the two. The gradient step size is further scaled as $\alpha/(1 + \mu)$, with $\alpha = 1$, to keep the update bounded as μ grows. Our absolute μ values therefore differ from ?’s in scale but preserve their role in the ADMM balance, as reflected in the $\mu \in [10, 100]$ action range (Section 3.3).

3.5 Baselines

The TFPnP reproduction is validated against five baseline methods. Three of the paper’s baselines (RED-CNN, LPD, RPGD) are either absent from LION or would require training beyond the project timeline, so were not included.

FBP applies a Ram-Lak filter followed by unfiltered backprojection through LION’s operator, with the analytical normalisation $\pi/(2N)$ for $N = 30$ projection angles. The result is clamped to non-negative values.

TV reconstruction uses ISTA with 200 iterations and anisotropic total variation at regularisation strength $\lambda = 0.01$. The data-fidelity gradient is normalised to unit maximum before each step, matching the ADMM z -step convention (Section 3.4), and non-negativity is enforced after each iteration.

DRUNet alone applies the natural-image DRUNet as a single denoising pass over the FBP reconstruction, with fixed $\sigma = 10$. This is not one of ?’s baselines, but provides a useful lower

bound, isolating how much of TFPnP’s performance comes from ADMM’s iterative refinement rather than the denoiser itself.

FBPConvNet [?] is a five-stage U-Net mapping sparse-view FBP reconstructions to full-view ground truth. Our FBPConvNet is trained end-to-end on the LIDC training split (250 patients across 80 epochs) using LION’s implementation, with MSE loss and Adam at learning rate 10^{-4} , at a fixed 5% sinogram noise level. This departs both from ?’s original protocol (clean projections at training, noise added at inference) and from ?’s FBPconv, whose near-constant PSNRs across noise levels (23.00, 22.94, 22.86 dB) implies mixed-noise training, although this is not explicitly stated. Fixed 5% training makes evaluation at 7.5% and 10% out-of-distribution, and is central to the reversal narrative in Section 4.1.

Fixed PnP-ADMM uses the same ADMM environment as TFPnP with hand-calibrated $\sigma = 1.5$ and $\mu = 20$, selected as the operating point maximising PSNR on a preliminary sweep. Any improvement of TFPnP over Fixed PnP-ADMM can be attributed to policy-based hyperparameter adaptation.

3.6 CT-specific DRUNet training

The headline TFPnP result (`run_04`) uses the natural-image DRUNet as denoiser prior, matching ?. As an extension, a CT-specific DRUNet is trained to test whether an in-domain denoiser improves reconstruction quality. The same architecture and checkpoint format as the pretrained natural-image DRUNet [?] is used, making it a drop-in replacement.

Training uses the LIDC training split (3300 slices) for 50 epochs with Adam at learning rate 10^{-4} , a cosine annealing schedule, and batch size 8. Each mini-batch samples a random noise level $\sigma \in [0, 50]$ per image, following ?’s multi-level convention, and loss is computed using mean squared error in the per-image $[0, 1]$ normalised space to match the DRUNet’s input convention. Validation PSNR at $\sigma \in \{5, 15, 25\}$ is monitored throughout training, with best-checkpoint selection based on PSNR at $\sigma = 15$. Figure 3.2 shows the training progression.

An initial drop-in test using the CT-DRUNet in place of the natural-image DRUNet within `run_04`’s TFPnP pipeline revealed a calibration mismatch, with PSNR collapsing to approximately 13 dB, below the FBP baseline. Diagnosis showed that the CT-DRUNet’s optimal σ sits at approximately 8, indicating that the regularisation scale at which the denoiser is most useful shifts with the training data distribution. When each denoiser is compared at its calibrated optimum, the CT-DRUNet outperforms the natural-image DRUNet at every noise level, with an advantage that grows with noise (Section 4.5).

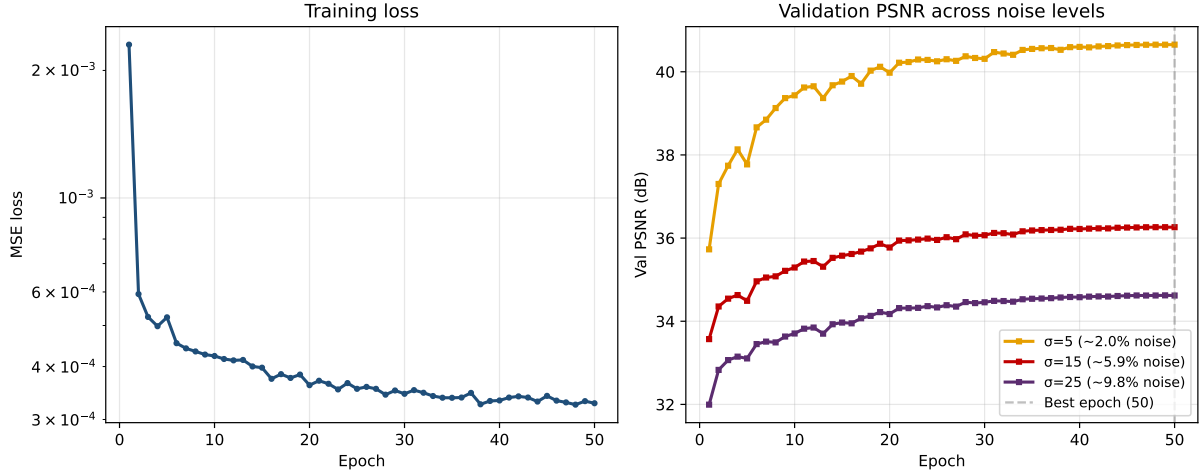


Figure 3.2: CT-DRUNet training curves. Left, MSE loss on the LIDC training split over 50 epochs. Right, validation PSNR at $\sigma \in \{5, 15, 25\}$. Best checkpoint selected by peak validation PSNR at $\sigma = 15$.

3.7 Compute infrastructure

All experiments were run on the Cambridge Service for Data Driven Discovery (CSD3) HPC cluster, using GPUs on the `ampere` partition. GPU access was allocated through a single account, `MPHIL-DIS-SL2-GPU`, which limited the number of ablation runs and rescue attempts feasible within the project timeline. All jobs were managed through SLURM, with a project-specific conda environment (`mphil_ct`) providing the LION toolbox, PyTorch, tomosipo, and supporting packages.

Table 3.1: Implementation choices differing from ?.

Aspect	?	Ours	Reason
CT geometry	30-view TorchRadon	30-view custom LION Geometry	LION had no matching preset
Detector count	182	724 ($\lceil 512\sqrt{2} \rceil$)	Cover image diagonal without truncation
Forward projector	TorchRadon	LION + tomosipo	LION native
Policy backbone stem	3×3	7×7 (torchvision default)	Stage widths and /32 downsampling unchanged; retraining to match was outside the compute budget
Value network	Policy feature extractor with weight norm + TReLU	Eight residual blocks at width 64, no normalisation, standard ReLU	Simpler BN-free critic
Policy input resolution	128×128	512×512 native slice	ADMM state passed at full resolution
Image intensity scale	$[0, 1]$ normalized	Native μ	Preserve physical units
z-step gradient norm	None	Normalized $A^\top r$	Compensate LION operator scaling
PSNR data range	Not stated (likely 1.0)	Per-image $x_{\text{gt.max}}()$	Matches data_range in LION conventions
PSNR / SSIM backend	Not specified	Custom PyTorch reimplementations of LION reference	Differentiability for ADMM step
HaarPSI backend	Not used	piq library	Compatible with μ -unit inputs
Policy σ range	Not stated	$[1, 5]$	Calibration for natural DRUNet
Policy μ range	Not stated	$[10, 100]$	Effective range post-gradient-normalization
Denoiser training data	BSD400 natural images	Same (run_04) LIDC (run_11)	In-domain extension
Denoiser σ range	$[1, 50]$	$[0, 50]$ (run_11)	Following ?
Denoiser loss	L_1	MSE	Simpler default, PSNR-aligned
Denoiser batch size	32	8	Fits GPU budget
Policy learning rates	$l_\theta=10^{-4}$, $l_\phi=5 \times 10^{-5}$ (later halved)	Critic 10^{-4} , policy 3×10^{-5} , param 10^{-6}	Empirical stability with our action ranges
Policy training data	PASCAL VOC natural images	LIDC (all runs)	In-domain policy training
FBPConvNet training noise	Mixed (implied by flat PSNR)	5% fixed	Simplest testable regime
Test set	COVID-19 lung, 50 images	LIDC-IDRI, 389 images	LION-native dataset

3.8 Evaluation metrics

Reconstruction quality is evaluated using peak signal-to-noise ratio (PSNR) [?], structural similarity index (SSIM) [?], and Haar wavelet-based perceptual similarity index (HaarPSI) [?]. PSNR is the standard metric used by ? and is essential for direct comparison to their reported results. The motivation for reporting SSIM and HaarPSI alongside PSNR is discussed in Section 5.3.

PSNR and SSIM are reimplemented in PyTorch to preserve differentiability through the reward computation chain, which is required for the model-based π_2 policy gradient’s backpropagation through the ADMM environment. Both reimplementations follow LION’s per-image convention using `data_range = x_{gt} .max(). HaarPSI is used only at evaluation time, so no custom implementation is required, allowing it to be loaded from the piq library [?], which accepts a configurable data_range compatible with our μ -scaled inputs. LION’s own HaarPSI implementation requires inputs strictly in $[0, 1]$ and was not used to avoid per-image normalisation. All metrics are computed per-image and averaged over the test set.`

3.9 Reproducibility difficulties

Several aspects of ?’s methodology were ambiguous or underspecified, requiring interpretation during reproduction. The paper does not explicitly state the PSNR data range convention, the baseline TV solver, the FBPconv training regime, or the ADMM z -step solver used for CT. The choices made in each case are documented in Table 3.1.

Reference code differences. The paper’s forward operator uses `TorchRadon` [?], which returns sinograms already scaled by the ray-length correction. LION’s tomosipo-based operator does not apply this correction, propagating the mismatch through the pipeline and requiring FBP normalisation, ADMM gradient scaling (the μ range), and the SCALE factor in noise generation (Section 3.2).

RL training stability. When the σ range was widened from $[1, 5]$ to $[1, 15]$ to accommodate the CT-trained DRUNet in `run_11`, the critic loss exploded around policy update step 15,000 and the policy σ output collapsed (Section 4.6). Our value network is not the one specified in Table 1 of ?, so this instability cannot be attributed to the paper’s formulation. Stabilising RL training with widened action ranges under our critic requires careful hyperparameter tuning (Section 5.2).

What worked well. ?’s reward function ($r = \Delta\text{PSNR} - \eta$), MDP formulation, and Algorithm 1 were sufficiently clear to reproduce directly, and the two-network policy–value architecture is straightforward to implement. The reproduction follows the paper’s algorithm closely and produces results within the paper’s reported range, subject to the architectural deviations recorded in Table 3.1.

Chapter 4

Evaluation

4.1 Headline reproduction result

The headline TFPnP result comes from `run_04_pat_250_e80`, where the policy is trained on 250 LIDC patients over 80 epochs, using the natural-image DRUNet as the denoiser prior and PSNR as the reward function. All evaluations use the full LIDC-IDRI test set (389 images) at three sinogram noise levels (5%, 7.5%, 10%). The full experiment configuration is documented in Section 3.3.

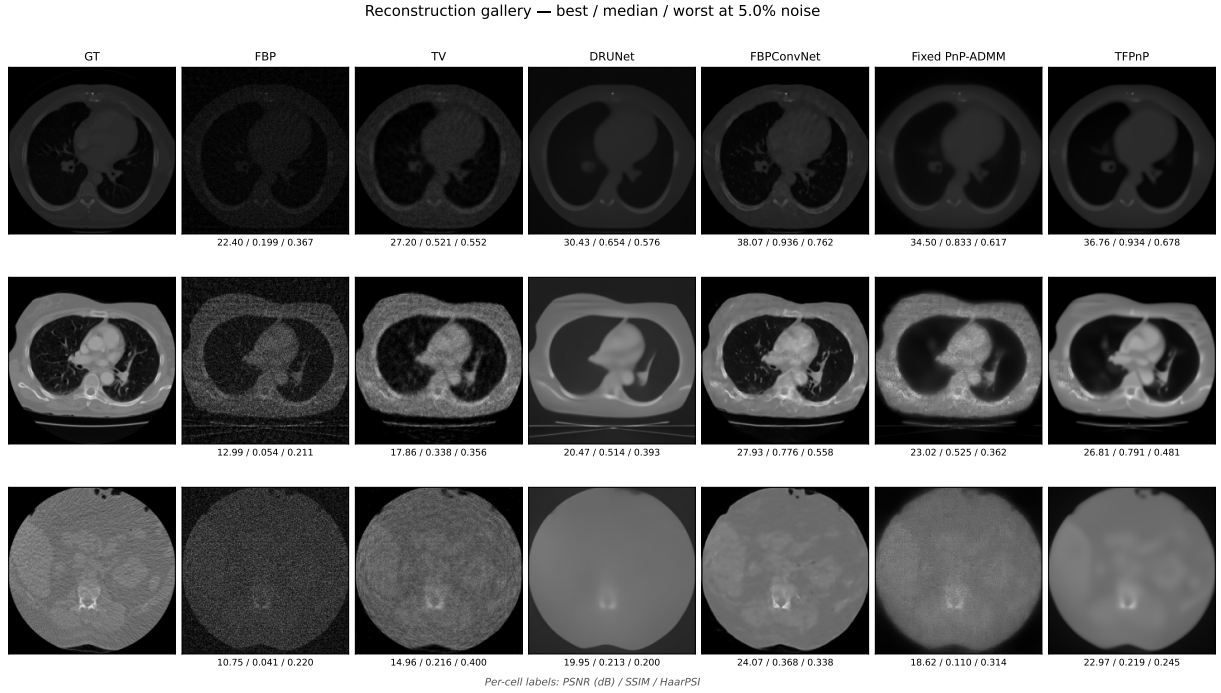
Six reconstruction methods are compared, including classical baselines (FBP and TV), the denoiser applied directly to the FBP reconstruction (DRUNet), an end-to-end learned network (FBPConvNet), and two hyperparameters, and the TFPnP reproduction with the learned policy). Table 4.1 reports mean PSNR, SSIM, and HaarPSI at each noise level.

The reproduction confirms the paper’s central claim, with TFPnP achieving **27.27 dB** PSNR at 5% noise, a +2.96 dB improvement over Fixed PnP-ADMM. Perceptual metrics (SSIM = 0.691, HaarPSI = 0.485) improve similarly over Fixed PnP-ADMM (0.499 and 0.405), confirming that the learned policy provides measurable value across all metrics.

One finding departs from the paper’s ranking. FBPConvNet reaches 28.58 dB at 5% noise, +1.31 dB above TFPnP, whereas ? report TFPnP ahead of FBPconv by +1.50 dB. The ranking inverts by 10% noise (Section 4.2), and the underlying causes are analysed in Section 5.1. Figure 4.1 shows a visual comparison of the best, median, and worst reconstructions.

Table 4.1: Reconstruction quality on the LIDC-IDRI test set (389 images). Best in **bold**, second-best underlined.

Method	5% noise			7.5% noise			10% noise		
	PSNR	SSIM	HaarPSI	PSNR	SSIM	HaarPSI	PSNR	SSIM	HaarPSI
FBP	14.56	0.075	0.233	13.65	0.058	0.223	13.22	0.052	0.216
TV	19.67	0.288	0.390	18.35	0.271	0.348	17.50	0.248	0.315
DRUNet ($\sigma=10$)	21.80	0.455	0.384	19.50	0.418	0.312	18.16	0.391	0.245
FBPConvNet	28.58	0.726	0.556	27.14	0.619	0.490	24.73	0.463	0.385
Fixed PnP-ADMM	24.31	0.499	0.405	22.42	0.376	0.336	21.50	0.340	0.290
TFPnP (Ours)	<u>27.27</u>	<u>0.691</u>	<u>0.485</u>	<u>26.58</u>	0.681	<u>0.443</u>	26.10	0.676	0.413

**Figure 4.1:** Reconstruction gallery for `run_04` at 5% noise. Rows: best, median, and worst test images by TFPnP PSNR. Per-cell labels give PSNR, SSIM, and HaarPSI.

4.2 Cross-noise behaviour

TFPnP’s practical value depends not only on its performance at the training noise level but also on how well it generalises across the noise range seen at inference. Figure 4.2 traces mean PSNR, SSIM, and HaarPSI as sinogram noise increases from 5% to 10%. TFPnP degrades gracefully, with PSNR dropping only -1.17 dB (from 27.27 to 26.10) and SSIM remains effectively flat (0.691 to 0.676). This robustness reflects the strength of the plug-and-play iterative approach, since the policy can adapt the denoising strength and penalty parameter to different noise levels, rather than producing a single fixed reconstruction.

By contrast, FBPConvNet degrades steeply across the same range (-3.85 dB PSNR, SSIM from 0.726 to 0.463), and TFPnP overtakes it on all three metrics at 10% noise. This is partly due to a methodological difference, as our FBPConvNet was trained at a fixed 5% noise level (Section 3.5) and extrapolates at higher noise. However, the crossover on perceptual metrics at 10% suggests

that the iterative structure of TFPnP provides a robustness that training-distribution matches alone would not explain, and is analysed further in Section 5.1.

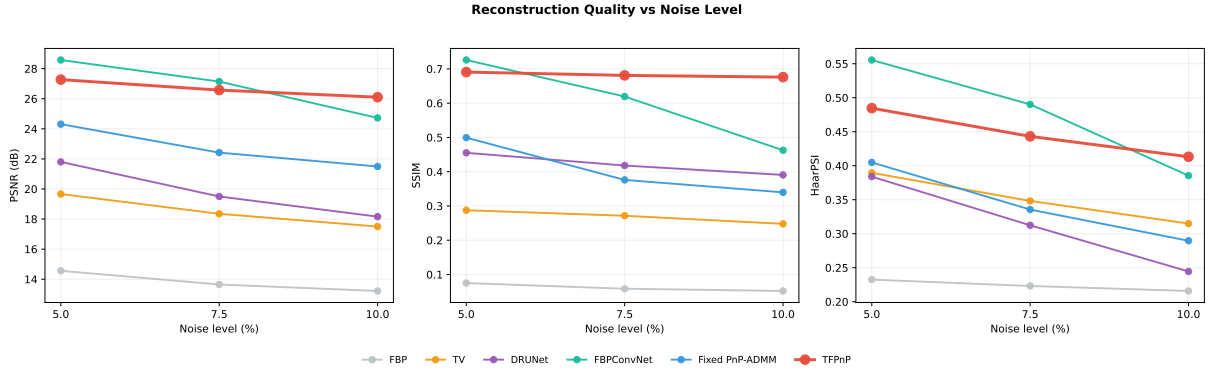


Figure 4.2: Mean PSNR, SSIM, and HaarPSI vs sinogram noise level. TFPnP overtakes FBPCConvNet on all three metrics at 10% noise.

4.3 Per-image variance

Averaging hides per-image variability. Figure 4.3 shows TFPnP’s three largest wins and losses over the next-best non-TFPnP method at 5% noise, ranked by per-image PSNR delta. TFPnP’s wins tend to occur on homogeneous slices with large uniform regions, while losses occur on complex slices with high-frequency detail such as multiple small nodules or intricate anatomy that doesn’t contain the lungs. Per-image variability (standard deviations of ~ 1.5 – 2 dB across all methods) means no method dominates uniformly, and reconstructions of comparable quality across methods are common. This is consistent with the general finding that PnP and end-to-end methods have complementary strengths, with neither uniformly outperforming the other.

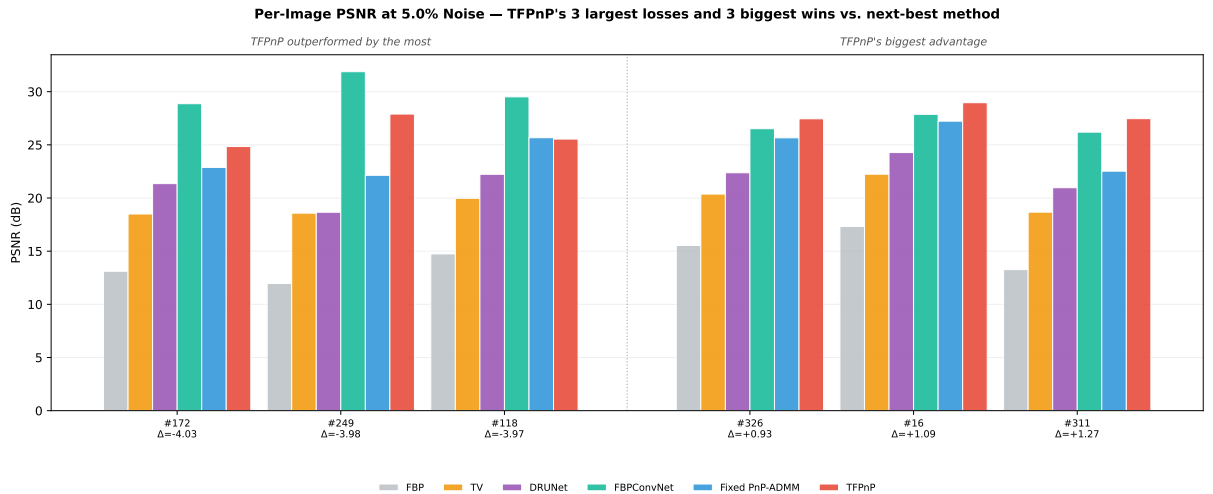


Figure 4.3: Per-image PSNR at 5% noise. TFPnP’s three largest wins (left) and losses (right) versus the next-best non-TFPnP method. Homogeneous slices favour TFPnP, complex ones favour FBPCConvNet.

4.4 Hyperparameter calibration sweeps

The Fixed PnP-ADMM baseline uses hand-calibrated $\sigma = 1.5$ and $\mu = 20$ values for the natural-image DRUNet, identified through sensitivity sweeps during notebook development. When the

denoiser is swapped for the CT-trained DRUNet developed in Section 3.6, this calibration is no longer valid, and Figure 4.4 and Figure 4.5 present calibration sweeps for both denoisers. The CT-trained DRUNet peaks at $\sigma = 8.0$ in the wrapper’s units; the interpretation of this apparent shift is revisited in ?? . The corresponding μ sweep at $\sigma = 8$ (see Figure 4.6) reveals a peak at $\mu = 10$, half the natural-DRUNet value.

Two limitations of these sweeps are worth noting. First, they were performed on a single representative slice at 5% noise, and a broader sweep across the validation set was intended but could not be completed within the available compute budget. Second, the sweep grids used $\sigma \in [1, 25]$ and $\mu \in [5, 50]$, but a finer grid around the peak could refine the calibration further.

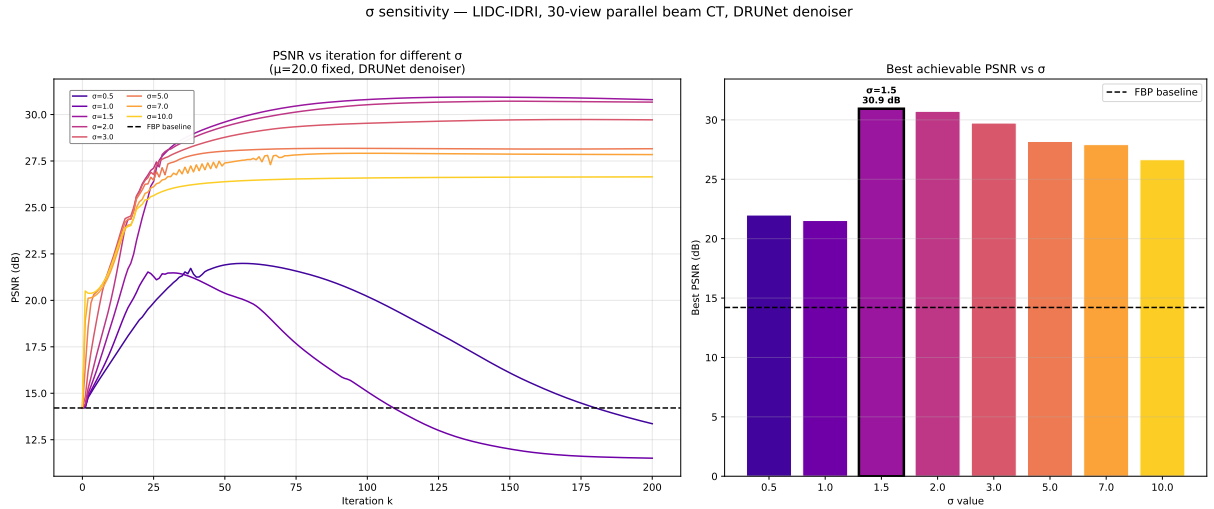


Figure 4.4: σ sensitivity for natural-image DRUNet within Fixed PnP-ADMM at 5% noise. Peak at $\sigma = 1.5$.

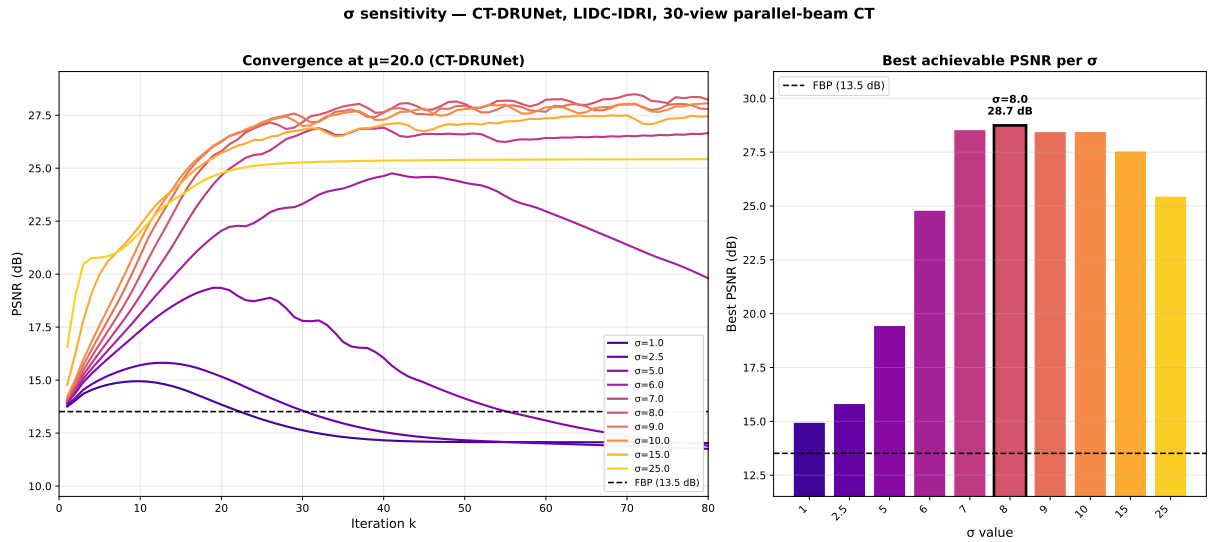


Figure 4.5: σ sensitivity for CT-trained DRUNet within Fixed PnP-ADMM at 5% noise. Peak at $\sigma = 8.0$.

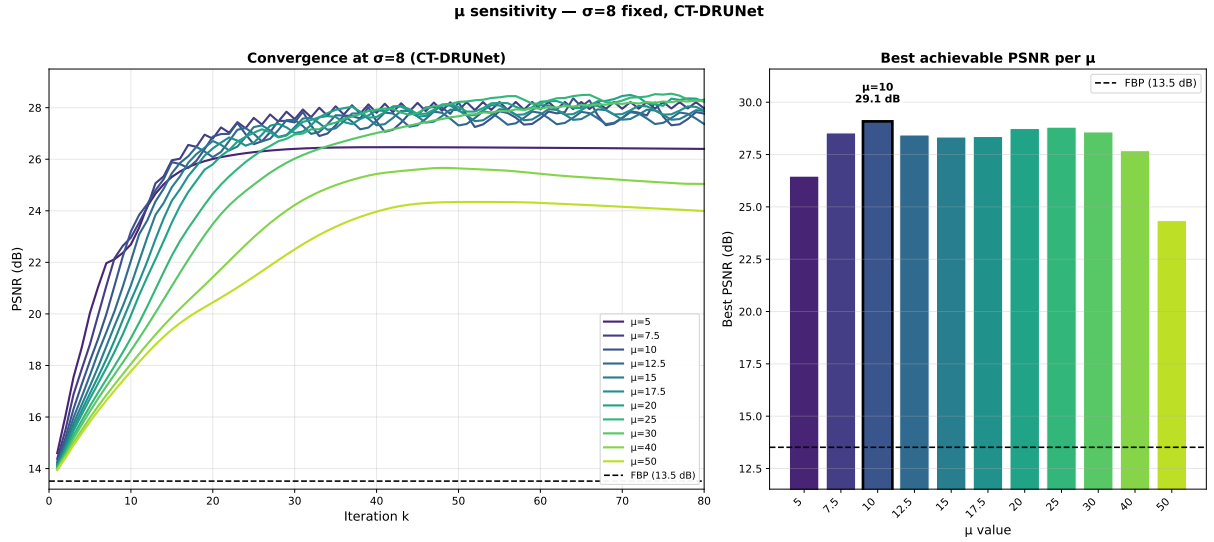


Figure 4.6: μ sensitivity for CT-trained DRUNet at $\sigma = 8$. Peak at $\mu = 10$.

4.5 In-domain denoiser experiment

With the CT-trained DRUNet calibrated to its own operating point ($\sigma = 8$, $\mu = 10$), a paired comparison of Fixed PnP-ADMM using each denoiser at its respective optimum was performed on the full 389-image test set (Table 4.2). The CT-DRUNet outperforms the natural-image DRUNet at every noise level, and the advantage grows with noise, from +0.71 dB at 5% to +1.98 dB at 10%. SSIM improvements follow the same trend, from +0.037 to +0.090.

The advantage grows with noise; its interpretation is discussed in Section 5.2. Figure 4.7 shows a visual comparison at all three noise levels.

This comparison uses Fixed PnP-ADMM rather than TFPnP with the learned policy, as retraining the policy against the CT denoiser (Section 4.6) encountered training instability that could not be resolved before a CSD3 cluster outage.

Table 4.2: Fixed PnP-ADMM at each denoiser’s calibrated optimum.

Denoiser	5% noise		7.5% noise		10% noise		Advantage	
	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM
Natural DRUNet ($\sigma=1.5$)	28.06	0.752	26.52	0.695	25.41	0.652	—	—
CT DRUNet ($\sigma=8$)	28.77	0.789	27.91	0.768	27.39	0.742	+0.71 to +1.98	+0.037 to +0.090

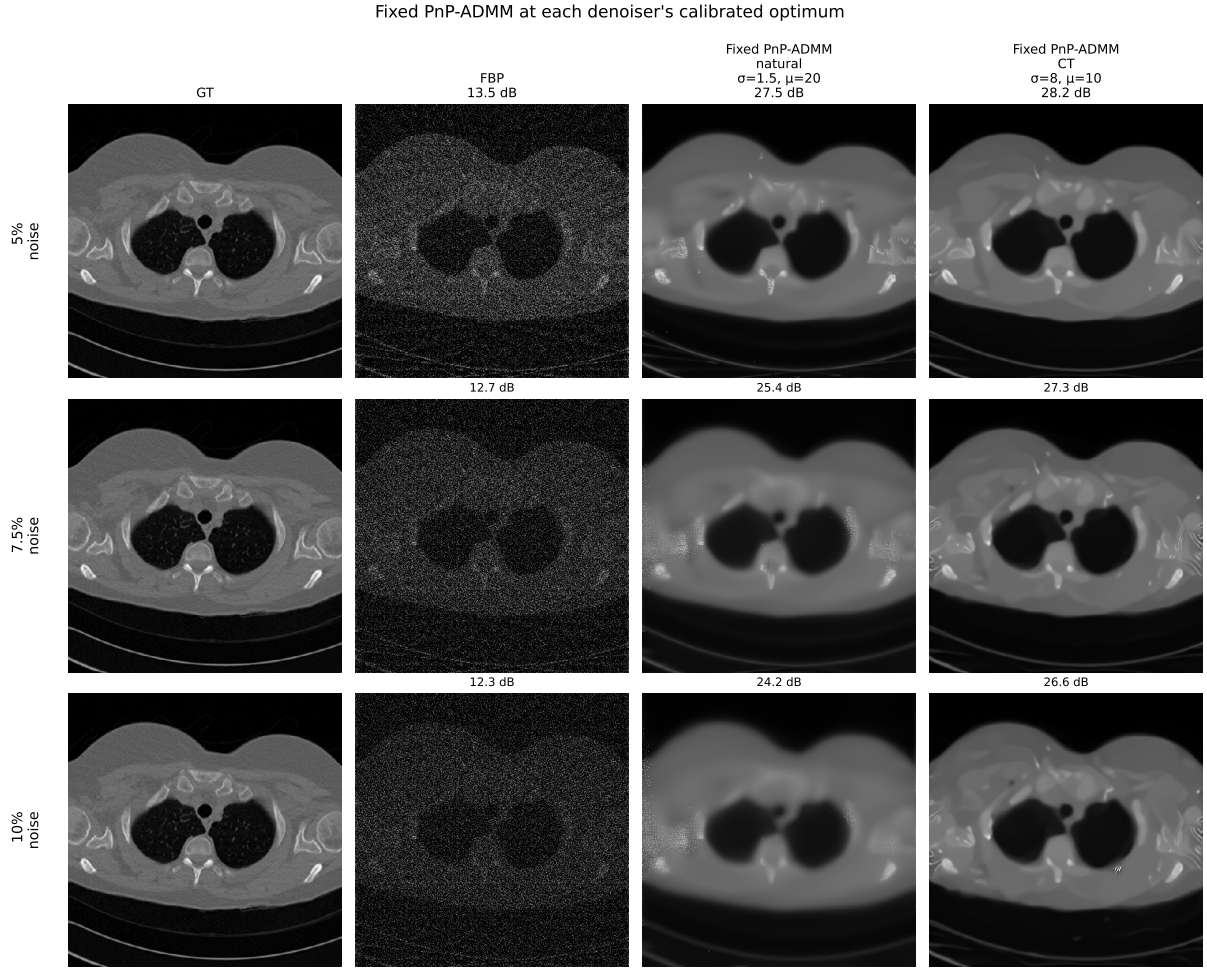


Figure 4.7: Fixed PnP-ADMM with denoisers at their calibrated optimum, at 5%, 7.5%, and 10% noise.

4.6 TFPnP policy retrain with CT-DRUNet

The natural next step is retraining the TFPnP policy against the CT-DRUNet, with a widened σ action range $[1, 15]$ to cover the new optimum and a narrowed μ range $[5, 50]$ chosen to keep the ADMM balance stable at the higher σ values. This run (`run_11`) underperformed `run_04`, reaching only 26.51 dB at 5% noise, well below `run_04`'s 27.27 dB and below even the Fixed PnP-ADMM baseline with CT-DRUNet at 28.77 dB (Section 4.5).

Figure 4.8 shows the diagnostic training curves. Around policy update step 15,000, the critic loss explodes to 10^{19} -scale magnitudes, the policy's mean σ output drifts from ~ 14 to ~ 9 , and both training and validation PSNR collapse from their earlier trajectory. The best checkpoint is saved from epoch 1, indicating no improvements were made. Since both the σ and μ ranges were changed simultaneously, the failure cannot be attributed to either alone, but the drift in the policy's mean σ output suggests the widened action space is the more likely cause. A third factor cannot be excluded. Our value network omits the weight normalisation and TReLU that Table 1 of ? specifies for the critic (Section 3.3), and both are adopted in their source papers for their stabilising effect on training. Whether the divergence follows from the widened action range, from this architectural omission, or from both, could not be separated at the time. A post-hoc diagnosis identifying a further, more fundamental cause is given in ??.

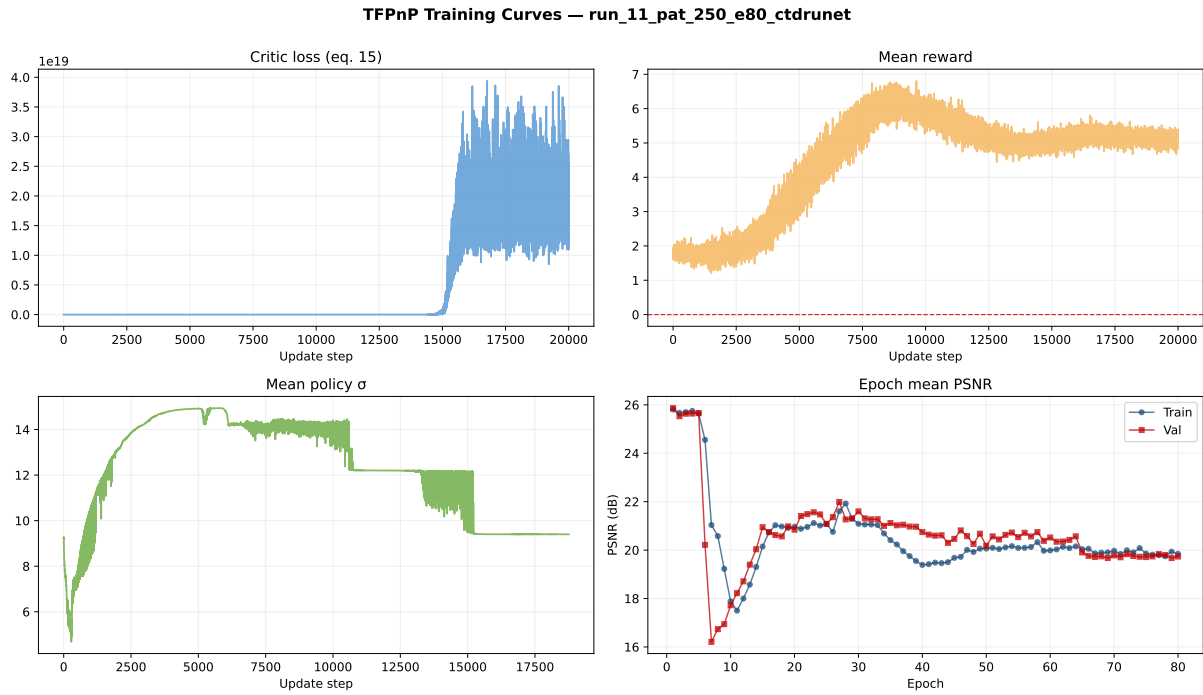


Figure 4.8: Training curves for run_11.

4.7 Reward shaping ablations

Four variant runs were carried out to test whether alternative reward signals could improve metric-specific performance, motivated by the finding of ? that PSNR correlates poorly with perceived image quality in medical imaging, raising the question of whether an RL policy rewarded on different metrics would produce better reconstructions than one trained on PSNR alone. The study is an extension undertaken in addition to the reproduction, rather than a component of ?’s original methodology, and results are summarised in Table 4.3.

The reward coefficients compensate for the difference in numeric scale between metrics. A typical per-step Δ PSNR is on the order of 0.1–1.0, whereas Δ SSIM and Δ HaarPSI are on the order of 0.001–0.01, so without scaling, they would be roughly 100 $\times$ smaller than the PSNR term and contribute negligibly to the policy gradient. The mixed rewards (run_07, run_08) therefore use $\alpha = 5$ to bring the perceptual term to a comparable order of magnitude as Δ PSNR, and the pure-replacement rewards (run_09, run_10) use $\alpha = 50$ so the overall reward magnitude remains comparable to run_04’s pure-PSNR reward.

The SSIM-only reward (run_09) achieves the highest PSNR and HaarPSI among all runs, and the mixed PSNR/SSIM reward (run_07) wins on SSIM specifically. In contrast, both HaarPSI-based rewards (run_08 PSNR/HaarPSI, run_10 HaarPSI only) underperform across all metrics. Given only a single training seed per configuration, some of this variation likely reflects RL training stochasticity rather than the differences between reward functions, so the PSNR reward chosen for run_04 remains a defensible default for direct comparison to the paper.

Table 4.3: Reward-shaping ablation at 5% noise. Best values in **bold**.

Run	Reward function	PSNR	SSIM	HaarPSI
run_04	PSNR (main)	27.27	0.691	0.485
run_07	PSNR + SSIM	27.18	0.707	0.481
run_08	PSNR + HaarPSI	26.47	0.640	0.437
run_09	SSIM only	27.32	0.695	0.489
run_10	HaarPSI only	26.76	0.658	0.460

Chapter 5

Discussion

5.1 Contextualising the TFPnP–FBPConvNet comparison

The reproduction confirms ?’s central claims. The learned policy improves on hand-calibrated hyperparameters by +2.96 dB at 5% noise (Section 4.1), and TFPnP degrades gracefully across the noise range (Section 4.2). The one direct disagreement with the paper is the FBPConvNet–TFPNP ranking at 5% noise, which reverses, but can be explained by two factors.

The first is an in-domain training asymmetry. In ?, TFPnP’s denoiser is trained on BSD400 and its policy on PASCAL VOC, both natural-image datasets. FBPconv’s training data is not stated explicitly, but the paper’s relatively flat FBPconv PSNRs across noise levels (23.00, 22.94, 22.86 dB) are consistent with out-of-domain training. In this reproduction, FBPConvNet is trained end-to-end on LIDC, while TFPnP’s policy is also LIDC-trained but the denoiser remains the natural-image DRUNet (Table 3.1). FBPConvNet is fully in-domain, yet TFPnP is only partially so, and at low noise the fully in-domain method extracts fine anatomical detail from the FBP reconstruction more effectively than the natural-image denoiser can.

The second is architectural. TFPnP overtakes FBPConvNet at 10% noise on all three metrics (Figure 4.2), with a PSNR drop across the range (−1.17 dB) roughly a third of FBPConvNet’s (−3.85 dB). Part of this reflects a training-distribution effect, as FBPConvNet was trained at fixed 5% noise while TFPnP saw the mixed regime, but the crossover on SSIM and HaarPSI is unlikely to be explained by that alone. Repeated denoising passes appear to remove sinogram noise progressively in a way a single forward pass cannot. The paper’s out-of-domain comparison obscured both effects, flattering TFPnP relative to FBPconv on the averaged ranking.

5.2 Does an in-domain denoiser close the gap?

If FBPConvNet’s advantage at low noise comes from its in-domain training, a natural extension is to give TFPnP the same advantage by training the denoiser on CT rather than natural images. The CT-DRUNet extension experiment (Section 3.6) yields two findings and one negative result. The first finding is that in the Fixed PnP-ADMM setting, the CT-trained DRUNet outper-

forms the natural-image DRUNet at every noise level, with the advantage growing with noise (Table 4.2). This result stands independently of what happens with the full TFPnP pipeline.

The second is that the CT-DRUNet operates at a fundamentally different σ scale than the natural-image DRUNet. The natural DRUNet peaks at $\sigma = 1.5$ (Figure 4.4), the CT-DRUNet at $\sigma = 8.0$ (Figure 4.5). Domain-specific training therefore does not just refine the denoiser’s ability to reject noise, but shifts the operating point at which the denoiser is most useful, with direct consequences for how policy σ ranges must be configured when swapping denoisers within TFPnP.

The negative result is that retraining the TFPnP policy against the CT-DRUNet (`run_11`) with a widened σ action range failed. The critic loss diverged and the policy σ output collapsed shortly afterwards. The obvious explanation is the action range itself, as widening σ from $[1, 5]$ to $[1, 15]$ enlarges the space of reachable ADMM states, and therefore the range of values the critic must learn to estimate. A second explanation is our own value network, as Table 1 of ? specifies that the critic uses the policy’s feature extractor with batch normalisation replaced by weight normalisation and ReLU replaced by TReLU. Our critic removes batch normalisation, as the paper does, but puts nothing in its place, using unnormalised convolutions and standard ReLU activations (Section 3.3). Weight normalisation is introduced specifically to stabilise training, but our critic has neither and diverges.

Taken together, these results suggest that the in-domain denoiser advantage is real and reproducible in the simpler Fixed PnP-ADMM setting, but that we were unable to translate it into the RL-based TFPnP pipeline. Whether that reflects a sensitivity of TFPnP’s RL formulation to action-range choices, or our own substitution of the value network, remains open.

5.3 On the choice of evaluation metrics

PSNR is the primary metric throughout this report, and was chosen for direct comparison to ?. However, PSNR captures only pixel-wise mean squared error, which does not always correlate with the visual quality in medical imaging. PSNR weights every pixel equally and is insensitive to the structure of errors, so a slightly blurred reconstruction that smooths away texture can score identically to one that preserves texture but adds a small amount of noise, although a radiologist would rate the two very differently. SSIM partially addresses the second issue by comparing local luminance, contrast, and structural patterns rather than only pixel intensities. HaarPSI extends this further through a Haar wavelet decomposition, and has been shown to correlate more strongly with radiological assessment than either PSNR or SSIM [?].

5.4 On direct comparison with ?

The absolute PSNR values reported in this project cannot be directly compared with those in ?, due to the differences detailed below.

First, the PSNR data-range convention differs, with this report following LION’s per-image

`data_range` = $x_{\text{gt}}.\text{max}()$ in native μ units, whereas the paper does not state its convention but most likely uses `data_range` = 1.0 on $[0, 1]$ -normalised images. This alone accounts for a ~ 5 dB shift in reported PSNR for the same mean squared error (Section 3.2), affecting direct comparisons but not within-experiment rankings.

Second, the paper reports on 50 randomly selected lung slices from the COVID-19 CT lung and infection segmentation dataset, and this project reports on the full 389-image LIDC-IDRI test set. The two datasets have different image statistics, noise characteristics, and anatomy distributions, so absolute PSNR is not directly comparable even at identical noise levels.

Third, the sinogram noise convention differs, as while both the paper and this reproduction follow a percentage-of-signal-standard-deviation convention, the paper’s scaling is not documented in detail, so replication is inexact (Section 3.2), but LION’s own more physical Poisson-Gaussian noise model was not used to mitigate this issue as much as possible.

Fourth, several implementation choices differ, consolidated in Table 3.1. The denoiser is trained with MSE loss (`train_drnet_ct.py`) rather than the L_1 loss used by ? and ?, chosen because PSNR is monotone in MSE and so aligns the denoiser with the RL reward ΔPSNR , at the cost of slightly more oversmoothing. The denoiser batch size was reduced to 8 instead of 32 to fit within the CSD3 GPU memory budget alongside the RL training runs. The policy learning rates are split into three groups (10^{-4} critic, 3×10^{-5} shared policy features and termination head, 10^{-6} parameter head; `train_tfpnp.py`) rather than the paper’s two-group schedule, reflecting the higher sensitivity of π_2 ’s model-based gradient through the differentiable ADMM environment, and no decay schedule was used in this reproduction. Finally, the μ action range is $[10, 100]$ rather than the paper’s $[0.01, 1]$, not as a hyperparameter choice but as a consequence of the z -step gradient normalisation (Section 3.4), since LION’s adjoint produces unnormalised backprojections $\sim 100\times$ larger than TorchRadon’s, so μ has to be $100\times$ larger to balance them.

Because of these differences, the reproduction’s central claim is the within-experiment finding that TFPnP outperforms Fixed PnP-ADMM by +2.96 dB at 5% noise, not the absolute agreement of numbers between the two studies. The FBPCNet-TFPnP ranking inversion at 5% noise is a similarly within-experiment finding, and its interpretation depends on the in-domain vs out-of-domain training asymmetry discussed in Section 5.1 rather than on absolute PSNR values.

Beyond these study-level differences, two limitations of the comparison itself should be noted. First, three of the paper’s six baselines (RED-CNN, RPGD, and LPD) are either absent from LION or would require training beyond the project timeline, so our comparison covers only a subset of the paper’s landscape. Second, all training runs used a single random seed due to compute constraints, so the reported numbers reflect one RL trajectory rather than an ensemble. Combined with the four factors above, this means the reproduction’s central claims are best interpreted as within-experiment findings rather than as absolute number-for-number reproductions of the paper’s results.

5.5 Broader implications

Three broader observations emerge from this reproduction.

First, ?’s fair-comparison methodology, in which all learning-based methods are evaluated on CT without domain-matched training data, may obscure the true capabilities of end-to-end methods on specific medical imaging domains. Allowing FBPCNN to train in-domain, substantially changes its performance relative to TFPnP. This does not invalidate the paper’s core contribution, but it does suggest that PnP methods need to be compared against domain-adapted baselines to inform practical deployment.

Second, TFPnP’s tuning-free claim is nuanced. It is accurate at inference time, but the training pipeline itself requires careful tuning of σ/μ action ranges, learning rates, and reward functions. The reward-shaping ablation (Section 4.7) shows meaningful differences across reward configurations, and the `run_11` failure illustrates that swapping in a different denoiser requires extensive re-engineering of the RL training pipeline. Tuning-free at inference does not mean tuning-free during development.

Finally, the same experimental structure, comparing in-domain and out-of-domain denoisers within a plug-and-play framework, could be applied to other PnP algorithms such as HQS, PnP-PGM, and RED. Whether the in-domain denoiser advantage transfers to these methods is an open question that this reproduction’s infrastructure would support with some additional work.

Chapter 6

Conclusion

This project reproduced the Tuning-Free Plug-and-Play method of ? within the LION toolbox [?] for 30-view parallel-beam sparse-view CT on LIDC-IDRI. The reproduction implements the RL policy-learning procedure, the ADMM environment, and the core baselines, and reproduces the paper’s central claim, as TFPnP achieves 27.27 dB at 5% noise, which is a +2.96 dB improvement over hand-calibrated Fixed PnP-ADMM. An upstream contribution to LION (pull request #184) fixes a bug in the FBPCnvNet integration surfaced during the reproduction.

The evaluation surfaces two findings the paper does not report. First, giving FBPCnvNet in-domain training reverses its ranking against TFPnP at low noise (+1.31 dB FBPCnvNet advantage at 5%), while TFPnP retains its advantage at 10% across all three metrics, an iterative-robustness effect obscured by the paper’s fully out-of-domain comparison. Second, a CT-specific DRUNet trained on LIDC improves Fixed PnP-ADMM by +0.71–+1.98 dB, with the advantage growing under noise, and operates at a $5\times$ higher optimal σ than the natural-image variant ($\sigma = 8$ vs $\sigma = 1.5$). Translating this gain into a fully retrained TFPnP policy encountered RL training instability. Our value network departs from the architecture specified by ?, so whether this instability is a property of TFPnP or of our implementation remains open.

Immediate follow-up work includes stabilising the CT-DRUNet policy retrain (narrower action range, gradient clipping, longer π_2 warmup, and restoring the weight-normalised TReLU value network of Table 1) and adding the RED-CNN, LPD, and RPGD baselines that were absent from LION, as well as retraining FBPCnvNet on a mixed-noise regime to complete the fair-comparison analysis. Broader extensions include applying the infrastructure to alternative plug-and-play algorithms such as half-quadratic splitting [?] and regularisation by denoising, and generalising the geometry to fan-beam or cone-beam CT via LION’s tomosipo backend. The wider lesson from this reproduction is that PnP methods demand careful denoiser calibration when the imaging domain shifts, and that reproducibility studies remain a valuable mechanism for surfacing methodological choices that primary papers leave underspecified.

Appendix A

Supplementary Material

A.1 Compute infrastructure

All experiments were run on the Cambridge Service for Data Driven Discovery (CSD3) HPC cluster, using GPUs on the **ampere** partition. GPU access was allocated through a single **MPHIL-DIS-SL2-GPU** SLURM account, which imposed a cap on the number of ablation and rescue runs that could fit within the project timeline. All jobs were managed through SLURM batch scripts, against a project-specific conda environment (**mphil_ct**, Python 3.11) providing the LION toolbox, PyTorch, tomosipo, and supporting packages.

Total compute across all experiments was approximately 290 GPU-hours: ~ 28 h per TFPnP training run across ten completed runs, plus ~ 2.5 h for CT-DRUNet training, ~ 0.5 h for FBP-ConvNet training, and the remainder split between hyperparameter sweeps and evaluation. A batched SLURM evaluation script (`slurm/full_eval_batch.sh`) reproduces all reported metrics from saved checkpoints in a single job.

A.2 Run matrix

Table A.1 summarises all eleven TFPnP training runs carried out during the project. **run_04** is the canonical reproduction run reported throughout Chapter 4, with all other runs varying one aspect at a time relative to it.

Table A.1: Summary of TFPnP training runs. Runs 04–12 use 250 LIDC patients, 80 epochs, and the paper’s default hyperparameters unless noted.

Run	Change vs. <code>run_04</code>	Rationale
<code>run_01</code>	20 patients, 20 epochs	Notebook-development sanity check that the pipeline runs and RL training converges
<code>run_02</code>	100 patients, 50 epochs	Scaling test to confirm behaviour holds beyond the 20-patient setting
<code>run_03</code>	Full LIDC split (3300 slices), 5 epochs	Test whether more patients with fewer epochs improves policy quality at similar total training budget
<code>run_04</code>	Main run: natural-image DRUNet, PSNR reward, $\sigma \in [1, 5]$, $\mu \in [10, 100]$	Direct reproduction of ?
<code>run_05</code>	σ floor lowered from 1.0 to 0.5	Test whether weaker denoising lets the policy fine-tune reconstructions in later ADMM iterations
<code>run_06</code>	$m = 3$, $N = 10$ (total ADMM steps unchanged)	Test sensitivity to the paper’s $m = 5$, $N = 6$ inner-outer split
<code>run_07</code>	Mixed PSNR/SSIM reward	Test whether a joint reward improves perceptual quality (Section 4.7)
<code>run_08</code>	Mixed PSNR/HaarPSI reward	Test HaarPSI-based reward against ?’s motivation
<code>run_09</code>	SSIM-only reward	Isolate the structural-similarity signal
<code>run_10</code>	HaarPSI-only reward	Isolate the perceptual-similarity signal
<code>run_11</code>	CT-DRUNet denoiser; $\sigma \in [1, 15]$; $\mu \in [5, 50]$	Test in-domain denoiser inside the full RL pipeline
<code>run_12</code>	CT-DRUNet; narrower action range; longer π_2 warmup	Rescue attempt for <code>run_11</code> , prepared but not submitted before the CSD3 outage; superseded by the recalibrated-denoiser approach of ??

A.3 Repository and artefacts

The full source for the reproduction is hosted at <https://gitlab.developers.cam.ac.uk/phy/data-intensive-science-mphil/assessments/projects/eaz21>.

The repository is organised as follows:

- `src/ct_tfpnp/` contains the Python package (policy network, value network, ADMM environment, denoiser wrappers, training loop, and baseline reconstructors)
- `scripts/` contains the training and evaluation scripts (`train_tfpnp.py`, `train_drnet_ct.py`, `train_fbpconvnet.py`, and the plotting and evaluation utilities).
- `slurm/` contains one SLURM launcher per training run (`train_tfpnp_02.sh` through `train_tfpnp_12.sh`) plus the baseline training and batch evaluation scripts.
- `notebooks/` contains the development narrative NB01–NB12, covering tomosipo and CT physics fundamentals, LION exploration, PnP-ADMM development, TFPnP paper analy-

sis, policy and environment prototyping, the full training pipeline, baselines, and the CT denoiser extension.

- **data/** contains a small LIDC subset for local smoke tests, with the full dataset loaded through LION’s dataloader on CSD3.
- **results/** contains the metrics CSVs, but per-run checkpoints are on CSD3 due to size.
- **figures/** contains the plots for each run and each notebook chapter.
- **report/** contains the LaTeX source, bibliography, and compiled PDF for this report.

Model checkpoints are too large to commit to the repository and live on CSD3, available on request:

- Natural-image DRUNet weights, pretrained from ? (**results/baselines/drunet_gray.pth**).
- CT-DRUNet best checkpoint (**results/baselines/drunet_ct/drunet_ct_best.pth**).
- FBPCnvNet checkpoint (**results/baselines/fbpconvnet/fbpconvnet_5pct_e80.pth**).
- TFPnP canonical checkpoint (**results/learned/run_04_pat_250_e80/best.pt**).

The upstream LION pull request (#184) contributing a fix for the FBPCnvNet integration is available at <https://github.com/CambridgeCIA/LION/pull/184>.

A.4 AI/LLM usage

Large language models (primarily Gemini and Claude) were used to support this project in the following areas:

- **Plotting boilerplate.** Matplotlib scaffolds for training curves, reconstruction galleries, and sensitivity sweeps.
- **Library usage.** Working out calling conventions for alignment with LION.
- **Debugging.** The CT-DRUNet calibration mismatch (Section 3.6), the **run_11** critic explosion (Section 4.6), and unit-scale mismatches between LION and TorchRadon (Section 3.4).
- **Docstrings and documentation.** Function-level docstrings for **ct_tfpnp** and generation of documentation using Sphinx.
- **Report review.** Redundancy checks, cross-referencing and typo fixes.
- **Cross-checking against the paper.** Comparing implementation choices with ? and consolidating differences into Table 3.1.

All model output was reviewed and, where used, edited before being committed to the repository or incorporated into this report. No model output was accepted without verification against the source paper, the LION toolbox, or empirical results from the reproduction itself. The core experimental design, hyperparameter choices, framing of the findings, and interpretation of the results are my own.

Bibliography

- Samuel G. Armato, Geoffrey McLennan, Luc Bidaut, Michael F. McNitt-Gray, Charles R. Meyer, Anthony P. Reeves, Binsheng Zhao, Denise R. Aberle, Claudia I. Henschke, Eric A. Hoffman, et al. The Lung Image Database Consortium (LIDC) and Image Database Resource Initiative (IDRI): A completed reference database of lung nodules on CT scans. *Medical Physics*, 38(2): 915–931, 2011.
- Ander Biguri, Subhadip Mukherjee, Xuzhi Zhao, Xi Liu, Xinyi Wang, Rui Yang, Yi Du, Yahui Peng, Mikael Brudfors, Mark Graham, Hyungon Ryu, Oliver Kutter, Andreas Hauptmann, Mustafa Al-Rubaye, Miika T. Nieminen, Mikael A. K. Brix, Austin Yunker, Rajkumar Ket-timuthu, John C. Roeske, Sasidhar Alavala, Subrahmanyam Gorthi, and Carola-Bibiane Schönlieb. Advancing the frontiers of deep learning for low-dose 3d cone-beam ct reconstruction. *IEEE Open Journal of Signal Processing*, 6:942–949, 2025. doi: 10.1109/OJSP.2025.3593189.
- Ander Biguri et al. LION: Learned Iterative Optimization Networks, 2024. URL <https://github.com/CambridgeCIA/LION>. Cambridge Image Analysis Group.
- Anna Breger, Ander Biguri, Malena Sabaté Landman, Ian Selby, Nicole Amberg, Elisabeth Brunner, Janek Gröhl, Sepideh Hatamikia, Clemens Karner, Lipeng Ning, et al. A study of why we need to reassess full reference image quality assessment with medical images. *Journal of Imaging Informatics in Medicine*, 38(6):3444–3469, 2025.
- Allard A. Hendriksen, Daniël M. Pelt, and K. Joost Batenburg. Tomosipo: fast, flexible, and convenient 3D tomography for complex scanning geometries in Python. *Optics Express*, 29(24):40494–40513, 2021.
- Kyong Hwan Jin, Michael T. McCann, Emmanuel Froustey, and Michael Unser. Deep convolutional neural network for inverse problems in imaging. *IEEE Transactions on Image Processing*, 26(9):4509–4522, 2017. doi: 10.1109/TIP.2017.2713099.
- Sergey Kastrulin, Jamil Zakirov, Denis Prokopenko, and Dmitry V. Dylov. Pytorch image quality: Metrics for image quality assessment, 2022. URL <https://arxiv.org/abs/2208.14818>.
- Jun Ma, Yixin Wang, Xingle An, Cheng Ge, Ziqi Yu, Jianan Chen, Qiongjie Zhu, Guoqiang Dong, Jian He, Zhiqiang He, Tianjia Cao, Yuntao Zhu, Ziwei Nie, and Xiaoping Yang. Toward data-efficient learning: A benchmark for covid-19 ct lung and infection segmentation. *Medical*

Physics, 48(3):1197–1210, 2021. doi: <https://doi.org/10.1002/mp.14676>. URL <https://aapm.onlinelibrary.wiley.com/doi/abs/10.1002/mp.14676>.

Rafael Reisenhofer, Sebastian Bosse, Gitta Kutyniok, and Thomas Wiegand. A haar wavelet-based perceptual similarity index for image quality assessment. In *Signal Processing: Image Communication*, volume 61, pages 33–43, 2018.

Matteo Ronchetti. Torchradon: Fast differentiable routines for computed tomography. *ArXiv*, abs/2009.14788, 2020. URL <https://api.semanticscholar.org/CorpusID:222067122>.

Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4): 600–612, 2004.

Kaixuan Wei, Angelica Aviles-Rivero, Jingwei Liang, Ying Fu, Hua Huang, and Carola-Bibiane Schönlieb. TFPnP: Tuning-free plug-and-play proximal algorithms with applications to inverse imaging problems. *Journal of Machine Learning Research*, 23(16):1–48, 2022.

Kai Zhang, Yawei Li, Wangmeng Zuo, Lei Zhang, Luc Van Gool, and Radu Timofte. Plug-and-play image restoration with deep denoiser prior. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):6360–6376, 2021.